Computer Organization

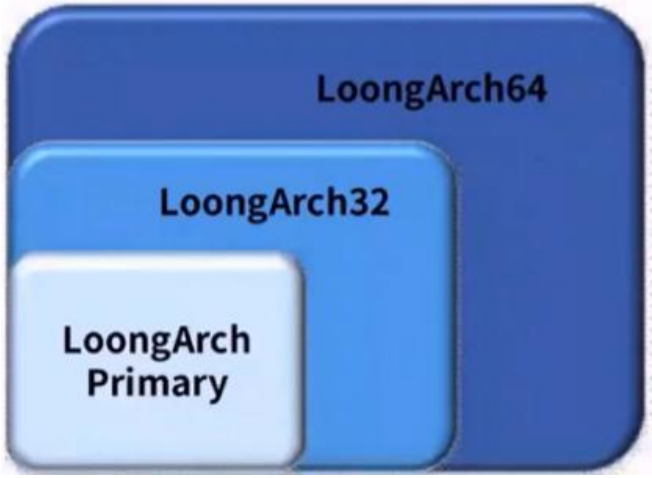
Lab8 LoongArch Primary

Topics

- **LoongArch Primary**
  - Comparison of Three RISC Instruction Formats(LoongArch, MIPS, RISC-V)
    - Data Type
    - Registers
    - Instruction format
    - Instruction
      - memory access
      - calculation
      - branch(conditional/ unconditional)
- **LARS**
  - LoongArch Assembly and Runtime Simulator, will assemble and simulate the execution of LoongArch assembly language programs.
- **Practice**

LoongArch Primary: for Teaching and Research in Higher Education Institutions

- The streamlined subset based on the 32-bit version of LoongArch - LoongArch Primary
  - User mode: Retains the most commonly used instructions in typical applications
    - Integer instructions: ~50 in total. Memory access instructions use only the "base address + offset" addressing mode.
    - Address alignment for floating-point instructions is optional; they can either be implemented fully or only the single-precision part can be achieved.
    - Atomic synchronization instructions are only LL/SC. Software maintenance instructions maintain data consistency with the data cache.
  - Core state: Supports mainstream Unix-like operating systems.
    - Only includes two privilege levels, PLV0 and PLV3.
    - Supports exceptions and interrupts, but they share the same entry point. Supports TLB MMU, with software responsible for TLB refills.
  - Software ecosystem: Maintains an independent small system.
    - QEMU, GCC ...
    - PMON, Linux kernel, busybox, buildroot.



Data Type

- 比特（bit，简记b）
- 字节（Byte，简记B，8bit）
- 半字（Halfword，简记H，16bit）
- 字　（Word，简记W，32bit）

	LA	MIPS I	RV32I
bit	bit	bit	bit
8bit	Byte, B	Byte, B	Byte, B
16bit	Halfword, H	Halfword, H	Halfword, H
32bit	Word, W	Word, W	Word, W

Registers

	LoongArch	MIPS I	RV32I
General-**R**egister	32 registers, each register has a 32-bit width. The 0th register always remains at 0.	32 registers, each register has a 32-bit width. The 0th register always remains at 0.	32 registers, each register has a 32-bit width. The 0th register always remains at 0.
PC	Independent of general-purpose registers, only modified by transfer instructions	Independent of general-purpose registers, only modified by transfer instructions	Independent of general-purpose registers, only modified by transfer instructions
Floating **R**egister	Independent of general-purpose registers	Independent of general-purpose registers	Independent of general-purpose registers

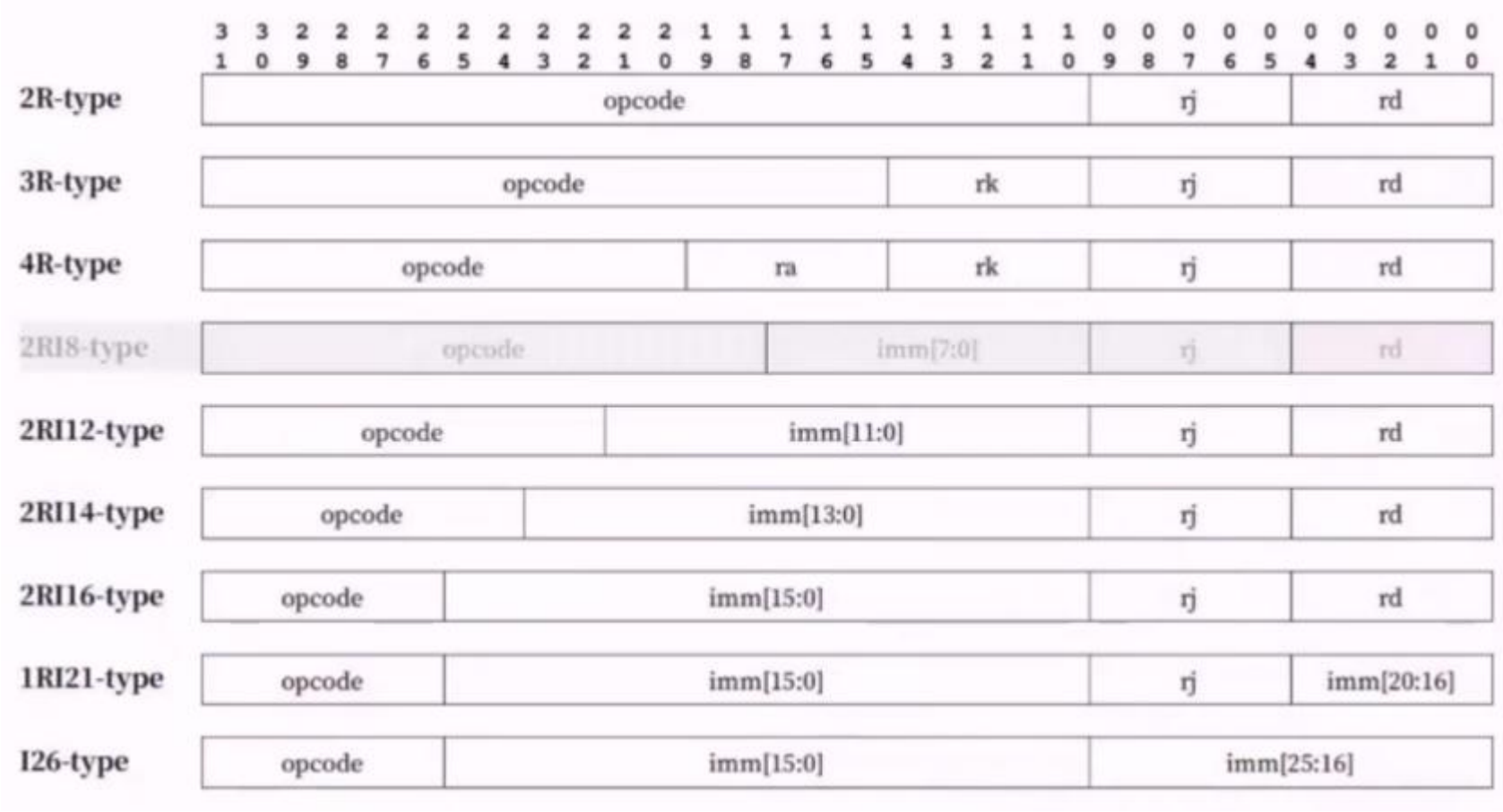
LA instruction format

- ✓ The machine codes of the instructions are all 32-bit wide.
- ✓ Instructions must be 4-byte boundary-aligned.
- ✓ 8 kinds of instruction formats
- ✓ **a single opcode field**
- ✓ register operand field(rd,rj,rk,ra)
  - ✓ rd- Usually serves as the destination register
  - ✓ rj- Usually serves as the source register 1
  - ✓ rk- Usually serves as the source register 2
  - ✓ ra- Usually serves as the source register 3

- ✓ Immediate number field.
  - ✓ 5 different length
  - ✓ Memory access and integer calculation: 12 bits
  - ✓ CSR addressing: 14bits
  - ✓ transfer instructions: 16bits, 21bits, 26bits

- ✓ All instructions have at most one destination register.

- ✓ The 3R-type with two sources and one destination is the most common instruction format; the 4R-type only appears in floating-point operations.

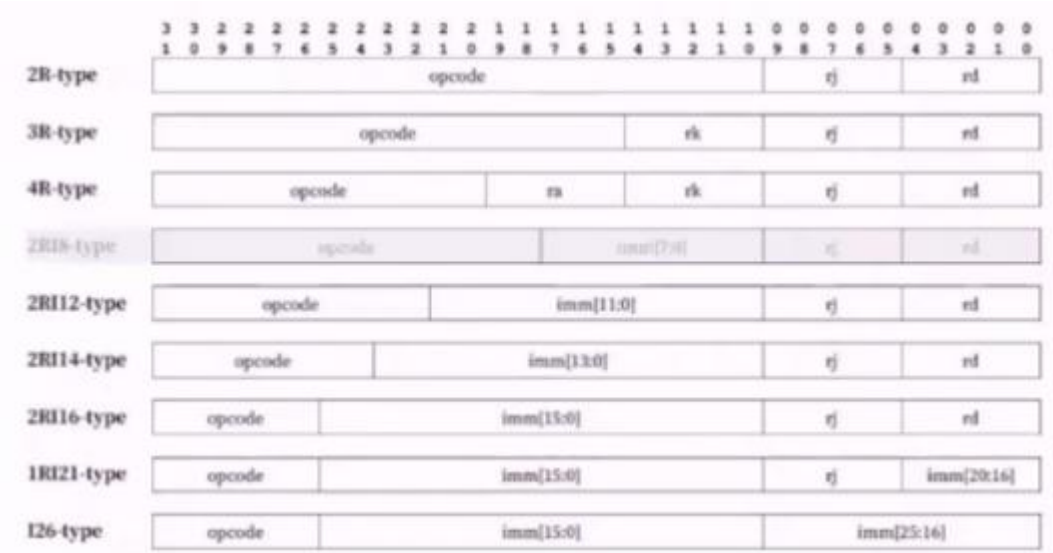


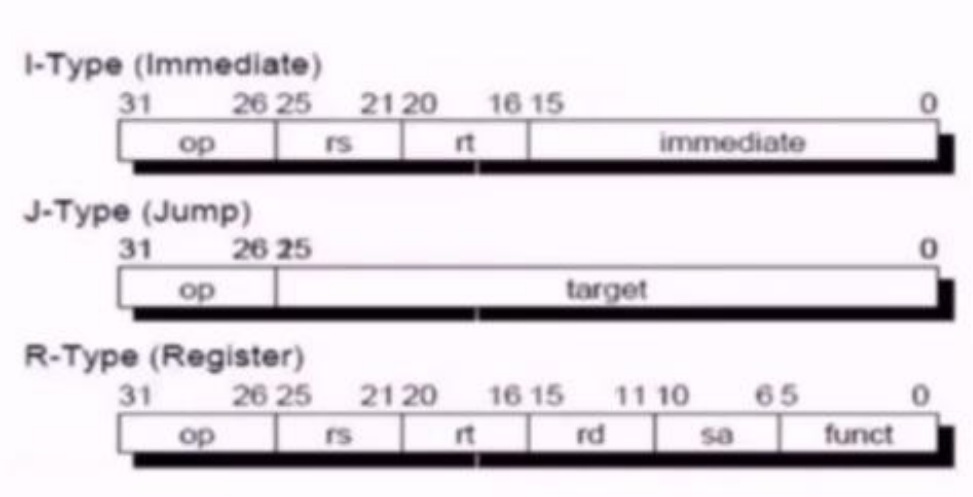
Example of LA Compact Instruction Format

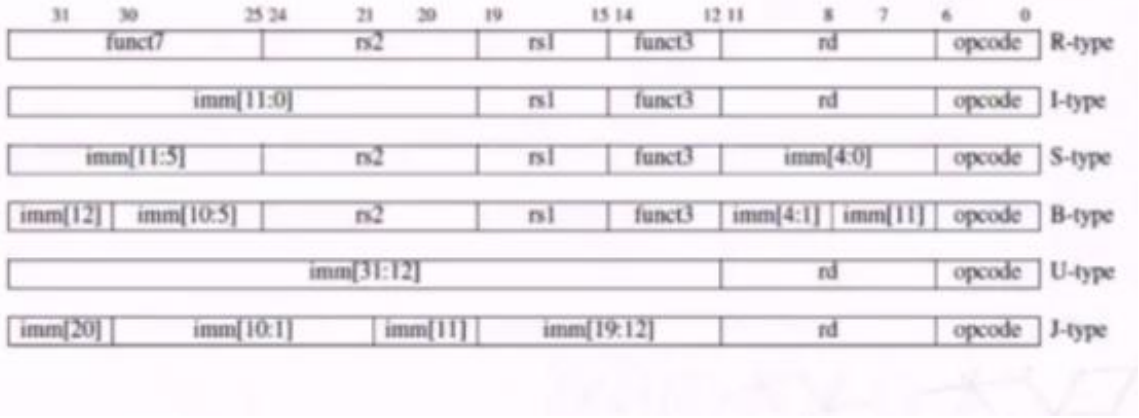
		31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0				
ADD.W	rd, rj, rk	0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0	rk	rj	rd	3R-type
FADD.S	fd, fj, fk	0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1	fk	fj	fd	3R-type
FABS.S	fd, fj	0 0 0 0 0 0 0 1 0 0 0 1 0 1 0 0 0 0 0 0 0 1		fj	fd	2R-type
SLTI	rd, rj, si12	0 0 0 0 0 0 1 0 0 0	si12	rj	rd	2RI12-type
CSRXCHG	rd, rj, csr	0 0 0 0 0 1 0 0	csr	rj!=0,1	rd	2RI14-type
FMADD.S	fd, fj, fk, fa	0 0 0 0 1 0 0 0 0 0 0 1	fa	fk	fj	fd	4R-type
LU12I.W	rd, si20	0 0 0 1 0 1 0	si20		rd	1RI20-type
LL.W	rd, rj, si14	0 0 1 0 0 0 0 0	si14	rj	rd	2RI14-type
BCEQZ	cj, offs	0 1 0 0 1 0	offs[15:0]	0 0 cj	offs[20:16]	1RI21-type
B	offs	0 1 0 1 0 0	offs[15:0]	offs[25:16]		I26-type
BEQ	rj, rd, offs	0 1 0 1 1 0	offs[15:0]	rj	rd	2RI16-type

Comparison of Three RISC Instruction Formats

- The three RISC instruction formats share the following common characteristics:
  - fixed length, alignment, and a small number of instruction formats;
  - opcode field, register field and immediate number field;
  - the three-register operand format of "two sources and one destination" is the most commonly used;
  - the positions where the register numbers to be read and written appear (basically) remain unchanged;
    - RV is the most thorough, while MIPS and LA have a few exceptions.







Memory access

龙芯中科 LOONGSON TECHNOLOGY

- **ld.b (load byte signed)** ld.b rd, rj, si12 GR[rd] = sext32(MEM[GR[rj]+sext32(si12)][7:0])
- **ld.bu (load byte unsigned)** ld.bu rd, rj, si12 GR[rd] = zext32(MEM[GR[rj]+sext32(si12)][7:0])
- **ld.h (load halfword signed)** ld.h rd, rj, si12 GR[rd] = sext32(MEM[GR[rj]+sext32(si12)][15:0])
- **ld.hu (load halfword unsigned)** ld.hu rd, rj, si12 GR[rd] = zext32(MEM[GR[rj]+sext32(si12)][15:0])
- **ld.w (load word signed)** ld.w rd, rj, si12 GR[rd] = MEM[GR[rj]+sext32(si12)][31:0]
- **st.b (store byte)** st.b rd, rj, si12 MEM[GR[rj]+sext32(si12)][7:0] = GR[rd][7:0]
- **st.h (store halfword)** st.h rd, rj, si12 MEM[GR[rj]+sext32(si12)][15:0] = GR[rd][15:0]
- **st.w (store word)** st.w rd, rj, si12 MEM[GR[rj]+sext32(si12)][31:0] = GR[rd][31:0]

LA	MIPS I	RV32I
ld.b rd, rj, si12	lb rt, offs16(rs)	lb rd, offs12(rs1)
ld.bu rd, rj, si12	lbu rt, offs16(rs)	lbu rd, offs12(rs1)
ld.h rd, rj, si12	lh rt, offs16(rs)	lh rd, offs12(rs1)
ld.hu rd, rj, si12	lhu rt, offs16(rs)	lhu rd, offs12(rs1)
ld.w rd, rj, si12	lw rt, offs16(rs)	lw rd, offs12(rs1)
st.b rd, rj, si12	sb st, offs16(rs)	sb rs2, offs12(rs1)
st.h rd, rj, si12	sh st, offs16(rs)	sh rs2, offs12(rs1)
st.w rd, rj, si12	sw st, offs16(rs)	sw rs2, offs12(rs1)

baiyh@sustech.edu.cn

Integer addition and subtraction operations

龙芯中科 LOONGSON TECHNOLOGY

- **add.w (add word)** add.w rd, rj, rk GR[rd] = GR[rj] + GR[rk]
- **sub.w (subtract word)** sub.w rd, rj, rk GR[rd] = GR[rj] - GR[rk]
- **addi.w (add immediate word)** addi.w rd, rj, si12 GR[rd] = GR[rj] + sext32(si12)

LA	MIPS I	RV32I
add.w rd, rj, rk	addu rd, rs, rt	add rd, rs1, rs2
sub.w rd, rj, rk	subu rd, rs, rt	sub rd, rs1, rs2
addi.w rd, rj, si12	addiu rs, rs, si16	addi rd, rs1, si12

注：

- sext32(A)表示将数值A符号扩展到32位宽。例如sext32(0x01)=0x00000001，sext32(0x80)=0xffffff80。

baiyh@sustech.edu.cn

Multiplication and division operations

龙芯中科 LOONGSON TECHNOLOGY

- **mul.w (multiply word signed)** mul.w rd, rj, rk GR[rd] = (GR[rj] $*_{signed}$ GR[rk])[31:0]
- **mulh.w (multiply high word signed)** mulh.w rd, rj, rk GR[rd] = (GR[rj] $*_{signed}$ GR[rk])[63:32]
- **mulh.wu (multiply high word unsigned)** mulh.wu rd, rj, rk GR[rd] = (GR[rj] $*_{unsigned}$ GR[rk])[63:32]
- **div.w (divide word signed)** div.w rd, rj, rk GR[rd] = GR[rj] $/_{signed}$ GR[rk]
- **mod.w (modulo word signed)** mod.w rd, rj, rk GR[rd] = GR[rj] $\%_{signed}$ GR[rk]
- **div.wu (divide word unsigned)** div.wu rd, rj, rk GR[rd] = GR[rj] $/_{unsigned}$ GR[rk]
- **mod.wu (modulo word unsigned)** mod.wu rd, rj, rk GR[rd] = GR[rj] $\%_{unsigned}$ GR[rk]

注：
- 取64位乘积低32位
- 取64位乘积高32位

LA	MIPS I	RV32M
mul.w rd, rj, rk	mult rs, rt	mul rd, rs1, rs2
mulh.w rd, rj, rk	mult rs, rt	mulh rd, rs1, rs2
mulh.wu rd, rj, rk	multu rs, rt	mulhu rd, rs1, rs2
div.w rd, rj, rk	div rs, rt	div rd, rs1, rs2
mod.w rd, rj, rk	div rs, rt	rem rd, rs1, rs2
div.wu rd, rj, rk	divu rs, rt	divu rd, rs1, rs2
mod.wu rd, rj, rk	divu rs, rt	remu rd, rs1, rs2

注：
- $*_{signed}$ 将两个操作数视作有符号数进行乘法；$*_{unsigned}$ 将两个操作数视作无符号数进行乘法；$/_{signed}$ 将两个操作数视作有符号数进行除法取商，$\%_{signed}$ 将两个操作数视作有符号数进行除法取余；$/_{unsigned}$ 将两个操作数视作无符号数进行除法取商，$\%_{unsigned}$ 将两个操作数视作无符号数进行除法取余。

daiyh@sustech.edu.cn

Integer comparison operation

龙芯中科 LOONGSON TECHNOLOGY

- **slt (set less than)** slt rd, rj, rk GR[rd] = GR[rj] $<_{signed}$ GR[rk]
- **sltu (set less than unsigned)** sltu rd, rj, rk GR[rd] = GR[rj] $<_{unsigned}$ GR[rk]
- **slti (set less than immediate)** slt rd, rj, si12 GR[rd] = GR[rj] $<_{signed}$ sext32(si12)
- **sltui (set less than unsigned immediate)** slt rd, rj, si12 GR[rd] = GR[rj] $<_{unsigned}$ sext32(si12)

LA	MIPS I	RV32I
slt rd, rj, rk	slt rd, rs, rt	slt rd, rs1, rs2
sltu rd, rj, rk	sltu rd, rs, rt	sltu rd, rs1, rs2
slti rd, rj, si12	slti rd, rs, si16	slti rd, rs1, si12
sltui rd, rj, si12	sltiu rd, rs, si16	sltiu rd, rs1, si12

注：

- $<_{signed}$ 将两个操作数视作有符号数进行小于比较；$<_{unsigned}$ 将两个操作数视作无符号数进行小于比较。

Logical bit operation

龙芯中科 LOONGSON TECHNOLOGY

- **and (and)** — and rd, rj, rk — GR[rd] = GR[rj] & GR[rk]
- **or (or)** — or rd, rj, rk — GR[rd] = GR[rj] | GR[rk]
- **nor (not or)** — nor rd, rj, rk — GR[rd] = ~(GR[rj] | GR[rk])
- **xor (exclusive or)** — xor rd, rj, rk — GR[rd] = GR[rj] ^ GR[rk]
- **andi (and immediate)** — andi rd, rj, ui12 — GR[rd] = GR[rj] & zext32(ui12)
- **ori (or immediate)** — ori rd, rj, ui12 — GR[rd] = GR[rj] | zext32(ui12)
- **xori (exclusive or immediate)** — xori rd, rj, ui12 — GR[rd] = GR[rj] ^ zext32(ui12)

LA	MIPS I	RV32I
and rd, rj, rk	and rd, rs, rt	and rd, rs1, rs2
or rd, rj, rk	or rd, rs, rt	or rd, rs1, rs2
nor rd, rj, rk	nor rd, rs, rt	N.A.
xor rd, rj, rk	xor rd, rs, rt	xor rd, rs1, rs2
andi rd, rj, ui12	andi rd, rs, ui16	andi rd, rs, si12
ori rd, rj, ui12	ori rd, rs, ui16	ordi rd, rs, si12
xori rd, rj, ui12	xori rd, rs, ui16	xori rd, rs, si12

注：

- zext32(A)将数值A零扩展到32位宽。例如zext32(0x01)=0x00000001，zext32(0x80)=0x00000080。

Shift opesration

龙芯中科 LOONGSON TECHNOLOGY

- **sll.w (shift left logic word)** — sll.w rd, rj, rk — GR[rd] = GR[rj] << GR[rk][4:0]
- **srl.w (shift right logic word)** — srl.w rd, rj, rk — GR[rd] = GR[rj] $>>_{logic}$ GR[rk][4:0]
- **sra.w (shift right arithmetic word)** — sra.w rd, rj, rk — GR[rd] = GR[rj] $>>_{arith}$ GR[rk][4:0]
- **slli.w (shift left logic immediate word)** — slli.w rd, rj, ui5 — GR[rd] = GR[rj] << ui5
- **srli.w (shift right logic immediate word)** — srli.w rd, rj, ui5 — GR[rd] = GR[rj] $>>_{logic}$ ui5
- **srai.w (shift right arithmetic immediate word)** — srai.w rd, rj, ui5 — GR[rd] = GR[rj] $>>_{arith}$ ui5

LA	MIPS I	RV32I
sll.w rd, rj, rk	sllv rd, rt, rs	sll rd, rs1, rs2
srl.w rd, rj, rk	srlv rd, rt, rs	srl rd, rs1, rs2
sra.w rd, rj, rk	srav rd, rt, rs	sra rd, rs1, rs2
slli.w rd, rj, ui5	sll rd, rt, sa	slli rd, rs1, shamt
srli.w rd, rj, ui5	srl rd, rt, sa	srli rd, rs1, shamt
srai.w rd, rj, ui5	sra rd, rt, sa	srai rd, rs1, shamt

注：

- $>>_{logic}$ 表示逻辑右移，$>>_{arith}$ 表示算术右移。

baiyh@sustech.edu.cn

Immediate loading

龙芯中科 LOONGSON TECHNOLOGY

- **lu12i (load upper from bit12 immediate)** lu12i rd, si20 GR[rd] = {si20, 12'b0}

LA	MIPS I	RV32I
lu12i rd, si20	lui rd, si16	lui rd, si20

- Combined with the "ori" instruction, it is used to store a 32-bit immediate value that does not fall within the range of [-2^{11}, $2^{12}-1$]
  - lu12i $t0, IMM[31:12]
  - ori $t0, $t0, IMM[11:0]

- MIPS adopts a high and low 16-bit split style. This is because the immediate numbers of its ordinary immediate arithmetic instructions are 16 bits wide.
- RV32I adopts a high 20-bit and low 23-bit split style. This is because the immediate numbers of its ordinary immediate arithmetic instructions are 12 bits wide.
  - RV32I instruction set cannot be simply implemented using the "lui+ori" combination, because the immediate value of the ori instruction is in fact represented using sign extension.

PC-based computing

龙芯中科 LOONGSON TECHNOLOGY

- **pcaddu12i (pc add upper from bit12 immediate)** pcaddu12i rd, si20 GR[rd] = PC + {si20, 12'b0}

LA	MIPS I	RV32I
pcaddu12i rd, si20	N.A.	auipc rd, si20

Combined with the jirl and load/store , used to complete the jump to the 32-bit relative offset address of the PC and memory access.

- pcaddu12i $t0, IMMH #Suppose the expected relative offset value of the PC is offset, and it is a multiple of 4.
- jirl $zero, $t0, IMML #OFFSET = {IMMH[19:0],12'b0} + sext32({IMML,2'b0}, 32}

- pcaddu12i $t0, IMMH #Suppose the expected relative offset value of the PC is offset.
- ld.b $t1, $t0, IMML #OFFSET = {IMMH[19:0],12'b0} + sext32(IMML, 32)

Conditional Branching

龙芯中科 LOONGSON TECHNOLOGY

- **beq (branch on equal)** — beq rj, rd, off16 — if (GR[rj]==GR[rd]) PC = TakenTgt
- **bne (branch on not equal)** — bne rj, rd, offs16 — if (GR[rj]!=GR[rd]) PC = TakenTgt
- **blt (branch on less than signed)** — blt rj, rd, offs16 — if (GR[rj]$<_{signed}$GR[rd]) PC = TakenTgt
- **bge (branch on greater than or equal signed)** — bge rj, rd, offs16 — if (GR[rj]$>=_{signed}$GR[rd]) PC = TakenTgt
- **bltu (branch on less than unsigned)** — bltu rj, rd, offs16 — if (GR[rj]$<_{unsigned}$GR[rd]) PC = TakenTgt
- **bgeu (branch on greater than or equal unsigned)** — bgeu rj, rd, offs16 — if (GR[rj]$>=_{unsigned}$GR[rd]) PC = TakenTgt

TakenTgt = PC + sext32({off16, 2'b0})

LA	MIPS I	RV32I
beq rj, rd, offs16	beq rs, rt, off16	beq rs1, rs2, offs12
bne rj, rd, offs16	bne rs, rt, off16	bne rs1, rs2, offs12
blt rj, rd, offs16	N.A.	blt rs1, rs2, offs12
bge rj, rd, offs16	N.A.	bge rs1, rs2, offs12
bltu rj, rd, offs16	N.A.	bltu rs1, rs2, offs12
bgeu rj, rd, offs16	N.A.	bgeu rs1, rs2, offs12

Unconditional relative PC jump

龙芯中科 LOONGSON TECHNOLOGY

- **b (branch)** b offs26 PC = BrTarget
- **bl (branch and link)** bl offs26 GR[1] = PC+4; PC = BrTarget

BrTarget = PC + sext32({off26, 2'b0})

LA	MIPS I	RV32I
b offs26	j target	jal zero, offs20
bl offs26	jal target	jal rd, offs20

Unconditional indirect jump

- **jirl (jump indirect register link)** jirl rd, rj, offs16 GR[rd] = PC+4;

  PC = GR[rj]+sext32({offs16, 2'b0})

LA	MIPS I	RV32I
jirl rd, rj, offs16	jr rs jalr rd, rs	jalr rd, rs1, offs12

Several commonly used indirect jump instructions are special forms of the jirl instruction.

1) Non-linking indirect jump: jr rj, which is actually jirl r0,rj,0; the most common function return is jr ra, which is jirl r0,r1,0.

2) Commonly used indirect function calls (returning with the jr ra instruction): jirl rj, which is actually jirl r1, rj, 0.

LoongArch ABI register usage convention

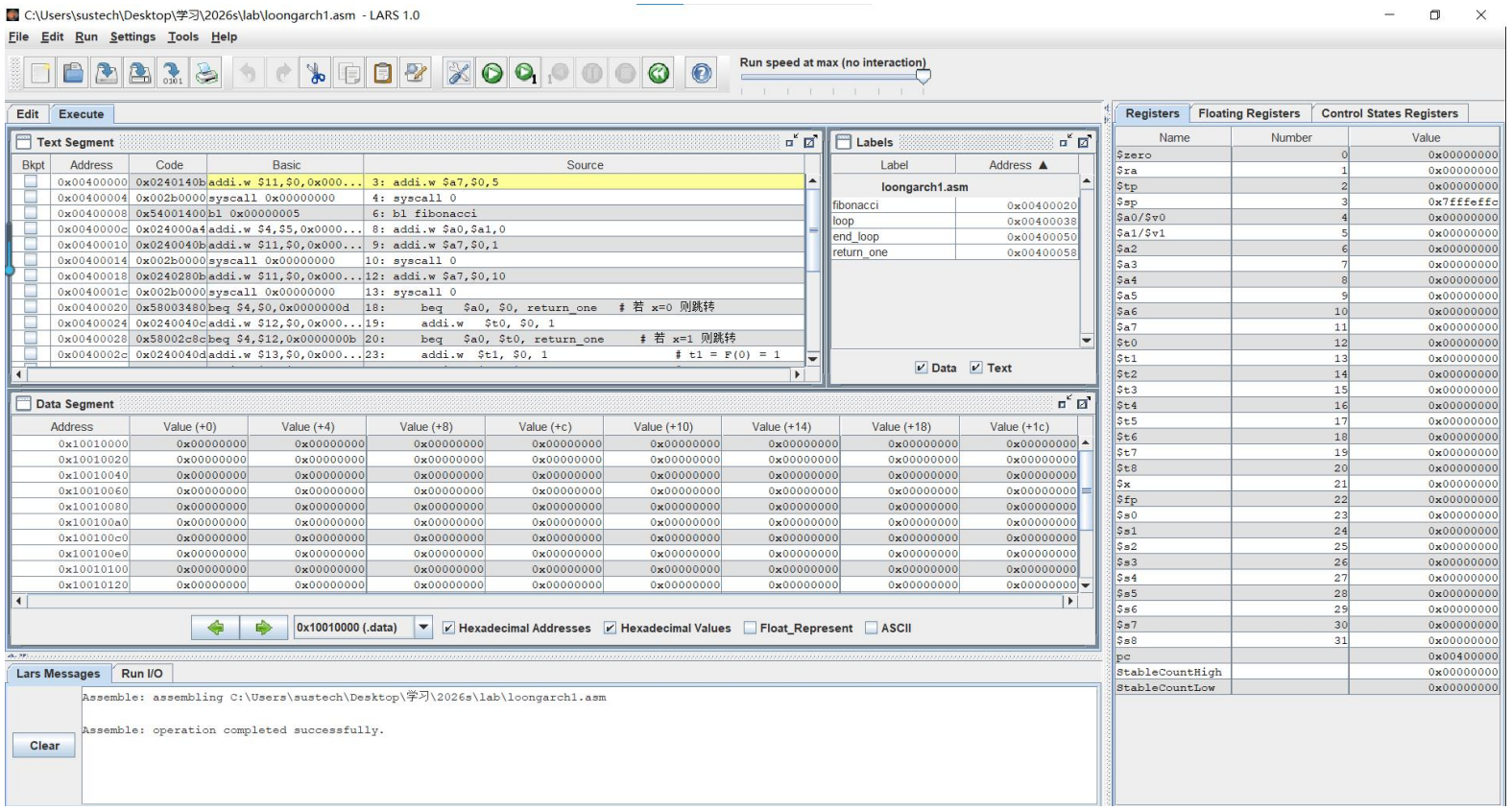
龙芯中科 LOONGSON TECHNOLOGY

- a0-a7: Function parameters;
- v0/v1: Function return values,
  - a0/a1 aliases;
- t*: Temporary variable, can be changed by sub-functions;
- s*: Variables that remain unchanged in sub-functions;
- ra: Return address;
- tp: Thread pointer;
- sp: Stack top pointer;
- fp: Stack frame pointer

寄存器号	助记符	Saver
0	zero	-
1	ra	caller
2	tp	-
3	sp	callee
4-11	a0-a7 v0/v1=a0/a1	caller
12-20	t0-t8	caller
21	reserved	-
22	fp	callee
23-31	s0-s8	callee

LARS

- **Lars: a lightweight assembly and runtime simulation tool specifically designed for the LoongArch Primary.**
  - It integrates **assembly**, disassembly, **simulated execution**, and **debugging** functions, and provides highly visualized monitoring of runtime **memory** and **register** status, greatly reducing the learning and debugging threshold for LoongArch assembly language.
- **Run (Plug and play, no installation required:** double-click to run**)**
- **The usage method of Lars is the same as that of Rars.**



Syscall in LARS

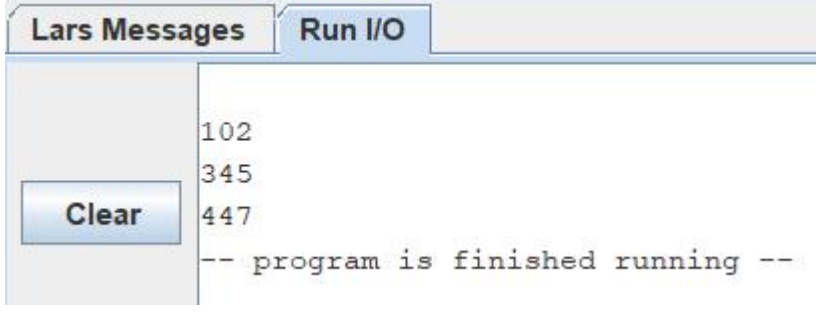
Basic Instructions | Extended (pseudo) Instructions | Directives | Syscalls | Exceptions | Macros

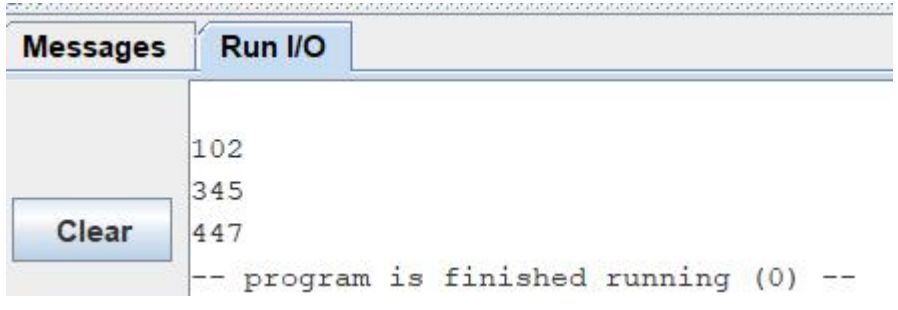
Service	Code in $a7	Arguments	Result
print integer	1	$a0 = integer to print	
print float	2	$f12 = float to print	
print double	3	$f12 = double to print	
print string	4	$a0 = address of null-terminated string to print	
read integer	5		$v0 contains integer read
read float	6		$f0 contains float read
read double	7		$f0 contains double read
read string	8	$a0 = address of input buffer $a1 = maximum number of characters to read	*See note below table*
sbrk (allocate heap memory)	9	$a0 = number of bytes to allocate	$v0 contains address of allocated memory
exit (terminate execution)	10		
print character	11	$a0 = character to print	*See note below table*
read character	12		$v0 contains character read

Demo 1 (LoongArch vs RISC-V)

```
.text                    #LoongArch
li.w $a7,5               #read operand1
syscall 0

add.w $t1, $0, $a0

li.w $a7,5               #read operand2
syscall 0

# operand1 + operand2

add.w $a0, $t1, $a0

li.w $a7,1               #output result
syscall 0

li.w $a7,10
syscall 0
```

```
.text                    #RISC-V
li a7,5                  #read operand1
ecall

add t1, x0, a0

li a7,5                  #read operand2
ecall

# operand1 + operand2

add a0, t1, a0

li a7,1                  #output result
ecall

li a7,10
ecall
```





Demo 2 (LoongArch vs RISC-V)

```
.data       #LoongArch
            str1:  .ascii  "Welcome "
            str2:  .ascii  "to "
            str3:  .asciz  "LA World"
.text
            la $t0, str2
            ld.b $t1, $t0, 0
            addi.w $t1, $t1, -32
            st.b $t1, $t0, 0

            la $a0, str1
            bl  print_string

            li.w $a7, 10
            syscall 0

print_string:
            li.w $a7, 4
            syscall 0
            jirl $0, $ra, 0
```

Lars Messages | Run I/O

```
Welcome To LA World
-- program is finished running --
```

```
.data       #RISC-V
            str1:  .ascii  "Welcome "
            str2:  .ascii  "to "
            str3:  .asciz  "RISC-V World"
.text
            la t0, str2
            lb t1, (t0)
            addi t1, t1, -32
            sb t1, (t0)

            la a0, str1
            jal  print_string

            li a7, 10
            ecall

print_string:
            li a7, 4
            ecall
            jalr x0, ra, 0
```

Messages | Run I/O

```
Welcome To RISC-V World
-- program is finished running (0) --
```

Demo 3 (LoongArch vs RISC-V)

```
//piece D3-1-1    LoongArch
.macro print_string(%str)
        .data
        pstr: .ascii %str

        .text
        la $a0,pstr
        li.w $a7,4
        syscall 0
.endm
```

```
//piece D3-1-2    LoongArch
.data
                sdata: .word 0xff7f7fff
                fneg1: .float -1
.text
                la $t0, sdata
                fld.s $f0, $t0, 0
                fmul.s $f12, $f0, $f0

                li.w $a7, 2
                syscall 0

                print_string("\n")

                la $t0, fneg1
                fld.s $f0, $t0, 0
                fmul.s $f12, $f0, $f0

                li.w $a7, 2
                syscall 0

                li.w $a7, 10
                syscall 0
```

Lars Messages | Run I/O

```
Infinity
1.0
-- program is finished running --
```

```
//piece D3-2-1    RISC-V
.macro print_string(%str)
        .data
        pstr: .ascii %str
        .text
        la a0,pstr
        li a7,4
        ecall
.end_macro
```

```
//piece D3-2-2    RISC-V
.data
                sdata: .word 0xff7f7fff
                fneg1: .float -1
.text
                la t0, sdata
                flw ft0, (t0)
                fmul.s fa0, ft0, ft0

                li a7, 2
                ecall

                print_string("\n")

                la t0, fneg1
                flw ft0, (t0)
                fmul.s fa0, ft0, ft0
                li a7, 2
                ecall

                li a7, 10
                ecall
```

Messages | Run I/O

```
Infinity
1.0
-- program is finished running (0) --
```

Practice

1. The Fibonacci sequence, also known as the golden ratio sequence, was introduced by mathematician Leonardo Fibonacci using rabbit breeding as an example, and is also known as the "rabbit sequence".

Its values are: 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Its definition: $F(0) = 1$, $F(1) = 1$, $F(n) = F(n-1) + F(n-2)$ （$n \geq 2$， $n \in N^*$）

Please use LA32R to complete the calculation about the Fibonacci sequence.

Input a integer m.

Use m as the index, and output the mth number in Fibonacci sequence.

The 0th number in Fibonacci sequence is 1, and the 1st number is 1, the 2nd number is 2, ……